

Data Structure (C Programming)

11/09/2026

Name : Harshad Sanjay Teli.

Roll No : 15

Batch : B1

Prerequisite

Based on .

Q1. Linked list implementation of stack

Input:

- Lab on DS: Linked List implementation of stack.
- 1. Linked list implementation of stack.
- 2. Write a C program to accept any expression from user and display it is valid or not.

↓

→

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Create a Node
```

```
struct Node {
```

```
    int data;
```

```
    struct Node *next;
```

```
};
```

```
struct Node *top = NULL;
```

```
// Push
```

```
void push(int value)
```

```
{
```

```
    struct Node *newNode = (struct Node *) malloc  
    (sizeof(struct Node));
```

```
newNode->data = value;
newNode->next = top;
top = newNode;
printf("%d pushed\n", value);
}
// Pop.
void pop()
{
    if(top == NULL) {
        printf("Stack Underflow\n");
    } else {
        struct Node *temp = top;
        printf("%d popped\n", temp->data);
        top = top->next;
        free(temp);
    }
}
//display
void display() {
    if(top == NULL) {
        printf("Stack is Empty\n");
    } else {
        struct Node *temp = top;
        printf("Stack: ");
        while(temp != NULL) {
            printf("%d->", temp->data);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}
```

```
int main()
{
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    pop();
    pop();
    return 0;
}
```

Output:

```
C:\Users\Harshad Teli\Downloads\DSA_LAB4>output
10 pushed
20 pushed
30 pushed
Stack: 30 -> 20 -> 10 -> NULL
30 popped
Stack: 20 -> 10 -> NULL
20 popped
10 popped

C:\Users\Harshad Teli\Downloads\DSA_LAB4>|
```

Q2) Write C program to accept any expression from user and display it is valid or not

Input:

2]



```
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char c) {
    top++;
    stack[top] = c;
}
void pop() {
    top--;
}
int main()
{
    char exp[MAX];
    int i, valid = 1;
    printf("Enter expression: ");
    scanf("%s", exp);
```

```
if(ch == '(' || ch == '{' || ch == '[') {
    push(ch);
}
else if (ch == ')' || ch == '}' || ch == ']') {
    if (top == -1)
    {
        valid = 0;
        break;
    }
    else
    {
        // check matching pair
        if ((ch == ')') && stack[top] == '(') ||
            (ch == '}') && stack[top] == '{') ||
            (ch == ']') && stack[top] == '[') {
            pop();
        }
        else
        {
            valid = 0;
            break;
        }
    }
}
if (valid == 1 && top == -1)
{
    printf("Expression is valid\n");
}
else
{
    printf("Expression is Invalid\n");
}
return 0;
}
```

Harshad Teli



Output:

```
C:\Users\Harshad Teli\Downloads\DSA_LAB4>gcc linked.c -o ./output  
C:\Users\Harshad Teli\Downloads\DSA_LAB4>output  
Enter expression: (a+b)*c  
Expression is INVALID  
C:\Users\Harshad Teli\Downloads\DSA_LAB4>|
```